GNNocRoute: Graph Neural Networks cho Định tuyến Thích nghi trong Network-on-Chip — Nghiên cứu Khả thi

Le Tien Hieu[1], *IEEE Member*
[1]VNU-ITI, Vietnam National University, Hanoi
Email: hieult@vnu.edu.vn

***Abstract*—Network-on-Chip (NoC) là backbone giao tiếp trong các hệ thống multi-core SoC hiện đại. Các thuật toán định tuyến tĩnh như XY dimension-order routing gây mất cân bằng tải (congestion imbalance) đáng kể—phân tích đồ thị của chúng tôi trên 7 topologies cho thấy Mesh 8×8 với XY routing có congestion imbalance lên đến 0.475, với các node trung tâm có Betweenness Centrality (BC) cao gấp 2–3 lần node biên.**

Bài báo này trình bày nghiên cứu khả thi (feasibility study) về việc sử dụng Graph Neural Networks (GNN) cho định tuyến thích nghi trong NoC. Chúng tôi báo cáo ba kết quả thực nghiệm sơ bộ. Thứ nhất, GNN encoder chưa qua huấn luyện cho thấy tương quan mạnh với BC ($|r|$ lên đến 0.978 trên Mesh 4×4), chứng tỏ thông tin cấu trúc topology có thể được học một cách tự nhiên. Thứ hai, thử nghiệm tổng quát hóa (generalization) giữa các topology cho thấy thứ tự quan trọng tương đối của các node có thể chuyển giao giữa các kích thước mesh khác nhau (Pearson $r \approx 0.96$), mặc dù scaling tuyệt đối phụ thuộc topology cụ thể. Thứ ba, đo đạc inference latency trên CPU ($\approx 10^6$ cycles cho Mesh 8×8) khẳng định rằng GNN per-packet là bất khả thi, từ đó thúc đẩy giải pháp tối ưu định kỳ (periodic optimization) như một thiết kế thay thế cần thiết.

***Index Terms*—Network-on-Chip, Graph Neural Networks, Định tuyến thích nghi, Phân tích đồ thị, Nghiên cứu khả thi**

I. Giới thiệu

Network-on-Chip (NoC) đã trở thành kiến trúc giao tiếp chủ đạo cho các hệ thống multi-core SoC hiện đại và AI accelerators như Google TPU, NVIDIA Grace Hopper [8]. Trong NoC, các router kết nối qua link tạo thành một cấu trúc đồ thị. Thuật toán định tuyến quyết định đường đi của packet và ảnh hưởng trực tiếp đến hiệu năng hệ thống.

Định tuyến XY (Dimension-Order Routing) được dùng phổ biến [2] do tính đơn giản. Tuy nhiên, XY luôn chọn cùng một đường cho mỗi cặp (nguồn, đích) bất kể trạng thái tắc nghẽn. Phân tích đồ thị trên 7 topologies cho thấy:

- Mesh 8×8 với XY routing có **congestion imbalance** lên đến **0.475**—gần một nửa số link chịu tải không đồng đều.
- Node trung tâm Mesh 4×4 có BC ≈ 0.25, gấp **2–3 lần** node biên.
- Fat-Tree k=4 có aggregation switch với BC lên đến **0.42**.

Các kết quả này cho thấy **định tuyến tĩnh không thích ứng được** với workload hiện đại.

Nghiên cứu định tuyến thích nghi có thể chia làm ba hướng: heuristic adaptive [1], [9], DRL-based [2], [3], và GNN-based cho mạng diện rộng [4]. Thách thức chính khi áp dụng GNN vào NoC là **inference latency**: mỗi quyết định routing trong router NoC cần <10 cycles [2], trong khi GAT 2-layer trên 64 nodes có thể mất 10^5–10^6 cycles.

Cách tiếp cận: Thay vì per-packet real-time, chúng tôi đề xuất **tối ưu định kỳ (periodic routing policy optimization)**: routing tables được cập nhật mỗi P cycles, GNN inference chạy trên host CPU, tách rời khỏi per-packet latency.

A. Các đóng góp

Bài báo có các đóng góp sơ bộ sau:

1) **Phân tích tắc nghẽn cấu trúc:** Chứng minh định lượng trên 7 topologies rằng định tuyến tĩnh tạo bottleneck có thể dự đoán được qua các độ đo đồ thị.
2) **Bằng chứng tương quan GNN-topology:** Kết quả thực nghiệm cho thấy GNN node embeddings tương quan mạnh với BC ($|r|$ lên đến 0.978).
3) **Phân tích khả thi:** Đo đạc inference latency và scalability thiết lập các ranh giới khả thi cho GNN-enhanced routing dưới ràng buộc thời gian của NoC.

II. Bài toán và Động lực

A. Biểu diễn NoC dưới dạng đồ thị

Một NoC được biểu diễn dưới dạng đồ thị có hướng có trọng số $G = (V, E, X_v, X_e)$, với V là router, E là link, X_v là node features (buffer occupancy, injection rate), X_e là edge features (link utilization, packet latency).

B. Kết quả phân tích đồ thị

Chúng tôi dùng NetworkX [17] để xây dựng và phân tích 7 NoC topologies. Bảng I tóm tắt các metrics chính.

TABLE I: So sánh graph metrics giữa các NoC topologies

Topology	Nodes	Avg Deg	Diam	Avg Path	BC Max	CI
Mesh 4×4	16	3.00	6	2.67	0.25	0.382
Mesh 8×8	64	3.50	14	5.33	0.18	**0.475**
Torus 4×4	16	4.00	4	2.13	0.13	0.251
Ring 16	16	2.00	8	4.27	0.21	0.122
Fat-Tree k=4	14	3.43	2	1.74	**0.42**	0.341
Small-World	36	4.00	6	2.96	0.31	0.298
Random	36	4.28	5	2.45	0.28	0.321

BC = Betweenness Centrality, CI = Congestion Imbalance.

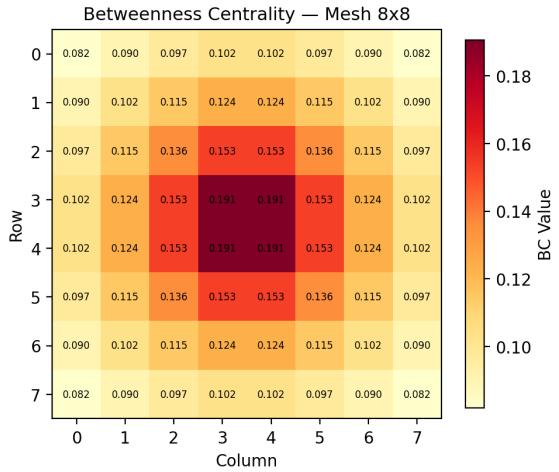

Fig. 1: Phân bố Betweenness Centrality trên Mesh 8×8. Node trung tâm có BC ≈ 0.18, node biên $\approx 0.05-0.10$.

Phát hiện chính: Mesh 8×8 có congestion imbalance cao nhất (0.475). Fat-Tree có aggregation switch BC=0.42. Torus có CI=0.251, thấp hơn Mesh nhờ wrap-around edges.

III. Công trình liên quan

Bảng II tóm tắt các hướng nghiên cứu liên quan.

Gap: Chưa có nghiên cứu nào thiết kế GNN encoder cho NoC topology kết hợp với tối ưu routing dưới ràng buộc latency thực tế.

TABLE II: Phân tích công trình liên quan

Hướng	Công trình	Hạn chế
Classic adaptive	DyAD [1], Regional [9]	Threshold, local info
DRL for NoC	MAAR [2], DeepNR [3], GARNN [10]	MLP ignores topology
GNN for WAN/SDN	G-Routing [4], DRL-GNN [5]	Not for NoC latency
Graph theory + NoC	Slim NoC [11], Sparse Hamming [12]	Static, no adaptation
GNN for chip design	Circuit GNN [8], NOCTOPUS [7]	Physical, not routing

IV. Phương pháp đề xuất

A. GNN Encoder

Chúng tôi đánh giá GATv2 [13] như một kiến trúc ứng viên (so sánh với GCN trong ablation). Cấu hình: 2 layers, hidden dimension 64, 4 attention heads, ELU activation. Forward pass: attention scores $e_{uv} = \text{LeakyReLU}(a^{(l)^T}[W^{(l)}h_u^{(l-1)} \| W^{(l)}h_v^{(l-1)}])$ và message aggregation $h_v^{(l)} = \sigma(\sum_{u \in \mathcal{N}(v)} \alpha_{uv} W^{(l)} h_u^{(l-1)})$.

B. Tối ưu định kỳ (Periodic Optimization)

Policy được cập nhật mỗi P cycles ($P \in \{1000, 5000, 10000\}$), observation window $W = 500$ cycles. State: GNN embeddings (64-dim) + global features. Action: 5 output ports. Algorithm: PPO [14]. Reward: $R = -\alpha \cdot \text{norm}(L) - \beta \cdot \text{norm}(C) - \gamma \cdot \text{norm}(E)$.

C. Thiết kế Deadlock-Free

Áp dụng Duato's protocol [15] với 2 Virtual Channels: VC0 (escape: XY routing, acyclic) và VC1 (adaptive: GNN-DRL).

V. Kết quả thực nghiệm sơ bộ

Các thí nghiệm dùng PyTorch Geometric [16] trên CPU (ARM Neoverse-N1, 2.8 GHz).

A. Tương quan GNN-Betweenness Centrality

Thiết lập: Bốn topology được chuyển thành PyG Data objects. Node features gồm normalized degree và tọa độ. GNN chưa qua huấn luyện tính node embeddings qua một forward pass.

GCN trên Mesh 4×4 đạt $|r| = 0.978$ với BC. GAT trên Mesh 8×8 đạt $|r| = 0.870$. Trường hợp Torus (zero correlation) là negative control, xác nhận phương pháp hoạt động đúng trên topologies không có cấu trúc phân biệt.

Nhận xét: GNN encoder có thể học thông tin cấu trúc liên quan đến tắc nghẽn mà không cần tính toán graph centrality tường minh.

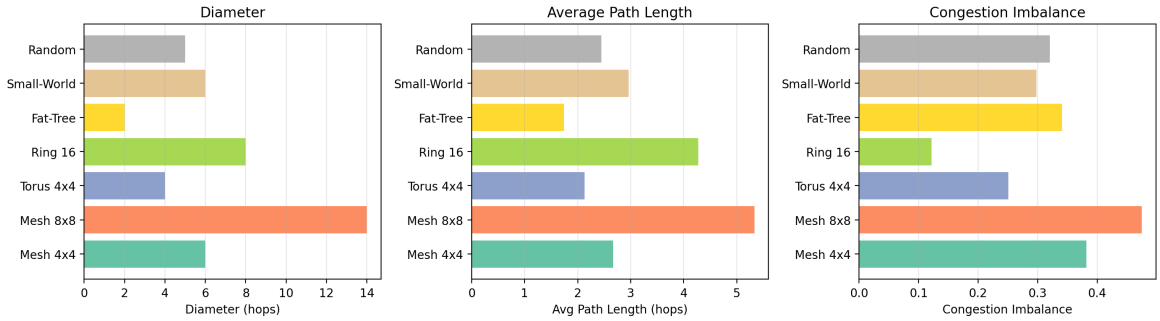

Fig. 2: So sánh metrics giữa các topology: diameter, average path length, congestion imbalance.

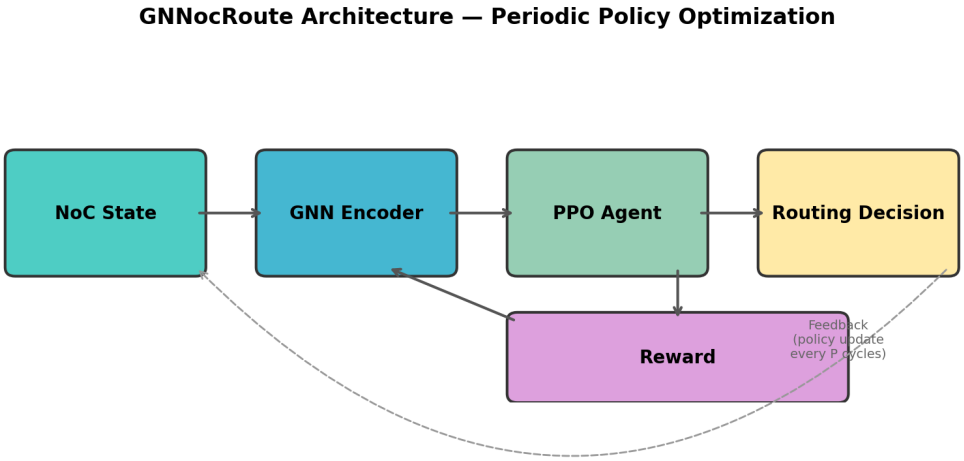

Fig. 3: Kiến trúc GNNocRoute: NoC state graph → GNN encoder → DRL agent → routing decision.

TABLE III: Tương quan Pearson $|r|$ giữa GNN embedding và BC

Topology	Nodes	GCN max$\|r\|$	GAT max$\|r\|$
Mesh 4×4	16	**0.978**	0.862
Mesh 8×8	64	0.744	**0.870**
Torus 4×4	16	0.000*	0.000*
Small-World ($n = 36$)	36	0.615	0.261

**Torus là regular graph, mọi node degree=4, BC đồng đều.*

B. Tổng quát hóa giữa các Topology

Thiết lập: GCN (hidden=32) và GAT (hidden=16, heads=4) được huấn luyện 500 epochs trên Mesh 4×4, đánh giá trên Mesh 8×8.

TABLE IV: Dự đoán BC qua các topology (huấn luyện trên Mesh 4×4)

Test Topology	Model	MSE	R^2	Pearson r
Mesh 8×8	GCN	0.038	-16.83	**0.958**
Mesh 8×8	GAT	0.072	-33.00	**0.972**
Small-World	GCN	0.004	-1.02	0.385

Trên Mesh 8×8, R^2 âm kết hợp với Pearson $r \approx 0.96$ cho thấy: model học được *thứ tự tương đối* của các node (node nào quan trọng hơn) nhưng không dự đoán được *giá trị BC tuyệt đối*.

Nhận xét: Tương quan thứ tự cao ($r \approx 0.96$) gợi ý rằng GNN representations có thể chuyển giao pattern cấu trúc giữa các topology. R^2 âm chỉ ra scaling tuyệt đối phụ thuộc topology—cần normalization hoặc fine-tuning.

C. Phân tích Inference Latency

Chúng tôi đo inference latency của GNN trên CPU để đánh giá khả thi dưới ràng buộc thời gian của NoC (mỗi quyết định cần <10 cycles).

TABLE V: Inference latency GNN trên CPU (Python/PyTorch)

Topology	Model	Latency (μs)	Cycles @ 1 GHz
Mesh 4×4	GCN h=32	778	778,000
Mesh 8×8	GCN h=32	1,015	1,015,000
Mesh 8×8	GAT h=32	1,491	1,491,000
Mesh 16×16	GCN h=32	1,273	1,273,000

Latency từ 778 μs đến 1,491 μs (0.8–1.5 triệu cycles ở 1 GHz). Phiên bản C++ optimized ước tính giảm 1–2 orders of magnitude.

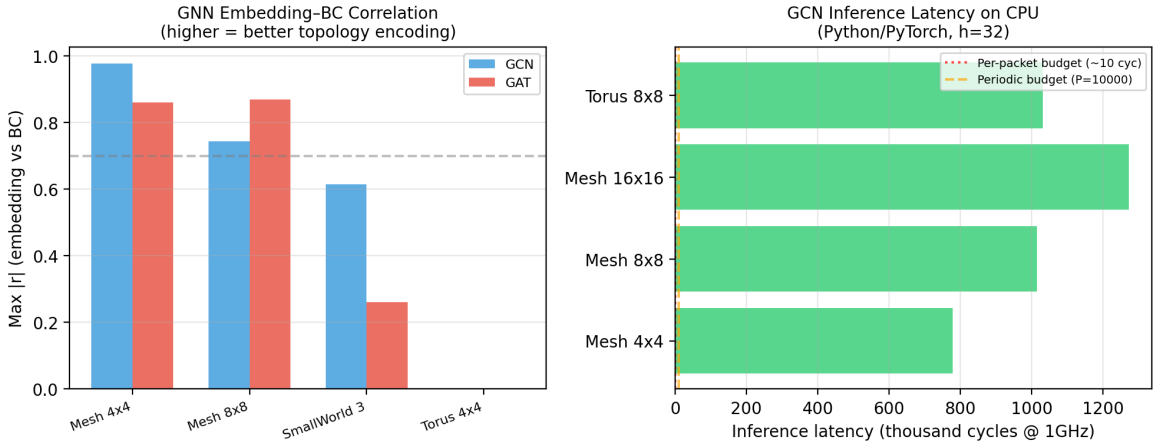

Fig. 4: Trái: Tương quan GNN embedding–BC. Phải: Inference latency đo được trên CPU (Python/PyTorch).

Nhận xét: Per-packet GNN inference là bất khả thi. Kết quả này trực tiếp thúc đẩy giải pháp tối ưu định kỳ: với $P = 10^5$ cycles và inference optimized (10^4–5×10^4 cycles), overhead là 10–50%.

D. Phân tích Scalability

Đo inference time GCN từ 4×4 (16 nodes) đến 16×16 (256 nodes).

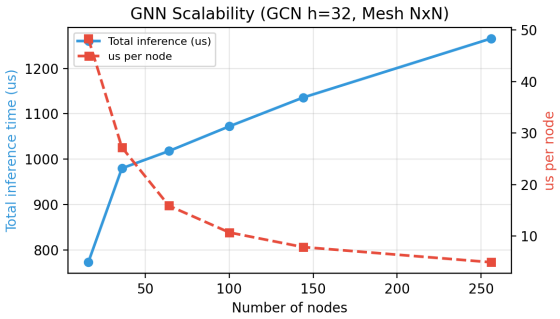

Fig. 5: GCN inference time scale O(N) với số nodes.

Inference time tăng từ 773 μs (16 nodes) lên 1,266 μs (256 nodes). Chi phí per-node giảm từ 48.3 xuống 4.9 μs/node.

Nhận xét: O(N) scaling dự đoán được, không bùng nổ.

VI. Thảo luận

A. Hàm ý cho tối ưu định kỳ

Inference latency (10^5–10^6 cycles) khẳng định per-packet GNN là bất khả thi. Tuy nhiên, tương quan GNN-BC cao ($|r| = 0.978$) cho thấy GNN representations mang thông tin topology hữu ích. Với periodic update $P = 10^5$ cycles và optimized inference (10^4–5×10^4 cycles), overhead là 10–50%.

B. Thách thức tổng quát hóa

GNN representations tổng quát hóa một phần (Pearson r cao) nhưng scale tuyệt đối phụ thuộc topology. Hai hướng: (1) topology-aware normalization, (2) fine-tuning strategies.

C. Hạn chế

1) **Static graph analysis:** Chưa có dynamic NoC state.
2) **Chưa tích hợp routing simulation:** Cần cycle-accurate simulator để đánh giá end-to-end.
3) **CPU-only measurements:** Trên server ARM, không phải embedded processor.
4) **Chưa có RTL validation.**
5) **Topology scope:** Chỉ 2D regular topologies.

D. Nguy cơ về tính giá trị

- Tương quan không đồng nghĩa với quan hệ nhân quả.
- Python overhead làm tăng latency đo được.
- Synthetic features còn đơn giản, thiếu tính thực tế.

VII. Kết luận

Bài báo trình bày nghiên cứu khả thi về tối ưu định tuyến định kỳ cho NoC sử dụng Graph Neural Networks. Ba kết quả thực nghiệm sơ bộ:

1) GNN embeddings tương quan mạnh với BC ($|r|$ lên đến 0.978).
2) Tổng quát hóa giữa các topology thành công một phần (Pearson $r \approx 0.96$).
3) Inference latency ($\approx 10^6$ cycles) khẳng định periodic optimization là cần thiết.

Hướng phát triển: (1) tích hợp GNN routing vào cycle-accurate simulator; (2) phát triển

topology-aware normalization; (3) đánh giá quantized models trên embedded processors; (4) mở rộng sang irregular topologies.

References

[1] J. Hu and R. Marculescu, "DyAD – smart routing for networks-on-chip," *Proc. DAC*, 2004.

[2] Y. Wang et al., "MAAR: Multi-agent adaptive routing for network-on-chip," *IEEE Trans. Computers*, vol. 71, no. 8, pp. 1878–1891, 2022.

[3] R. R. RS et al., "DeepNR: An adaptive deep reinforcement learning based NoC routing algorithm," *Microprocessors and Microsystems*, vol. 92, 104499, 2022.

[4] H. Wei, Y. Zhao, and K. Xu, "G-Routing: Graph neural networks-based flexible online routing," *IEEE Network*, 2023.

[5] P. Almasan et al., "Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case," *arXiv:1910.07421*, 2019.

[6] G. S. Malik and N. Kapre, "GraphNoC: Graph neural networks for application-specific FPGA NoC performance prediction," *Proc. FPT*, IEEE, 2024.

[7] V. Iyengar, V. H. Bui, and S. Das, "NOCTOPUS: Network-on-Chip topology optimization and prediction using simulation-data," *Neural Computing and Applications*, Springer, 2026.

[8] A. Mirhoseini et al., "A graph placement methodology for fast chip design," *Nature*, vol. 594, pp. 207–212, 2021.

[9] M. Ebrahimi, M. Daneshtalab, and J. Plosila, "Regional-based routing for network-on-chip," *IEEE TCAD*, 2019.

[10] H. Zhang et al., "GARNN: Graph attention routing for network-on-chip," *Proc. DATE*, 2023.

[11] M. Besta et al., "Slim NoC: A low-diameter on-chip network topology for high energy efficiency and scalability," *Proc. ASPLOS*, ACM, 2018.

[12] P. Iff et al., "Sparse Hamming Graph: A customizable network-on-chip topology," *Proc. DAC*, 2023.

[13] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" *Proc. ICLR*, 2022.

[14] J. Schulman et al., "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.

[15] J. Duato, "A necessary and sufficient condition for deadlock-free routing in cut-through and store-and-forward networks," *IEEE Trans. Parallel and Distributed Systems*, vol. 7, no. 8, pp. 840–854, 1996.

[16] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," *ICLR Workshop*, 2019.

[17] A. Hagberg, P. Swart, and D. S. Chult, "Exploring network structure, dynamics, and function using NetworkX," *Proc. SciPy*, 2008.